From Behavior Cloning to Offline Reinforcement Learning

Learning Better Decisions from Logged Data

Stephan Mandt July 14, 2026

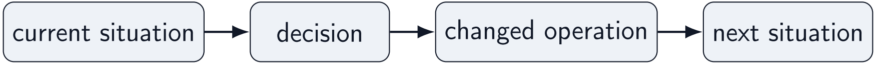
From Prediction to Decision

Prediction	**Decision**
What will happen?	What should we do?
Truck arrival time	Which truck to unload next
Grain quality	Where to route the grain
Train delay	Which loading schedule to use

Machine learning describes the operation. Decision-making changes it.

Reinforcement learning is a framework for learning such decisions.

Why Decisions Are Different



- Supervised learning learns from a fixed dataset.
- Decision-making selects actions rather than making predictions.
- Those actions influence future states and therefore the data seen later.

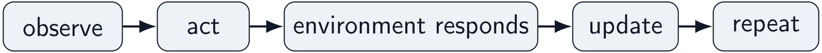
Prediction assumes a mostly stationary data source.
Decisions change the state distribution the model will face next.

How Have We Solved Decision Problems?

Rules and expert systems	Optimization and control	Reinforcement learning
Encode operational knowledge directly.	Model the system and compute good actions.	Improve decisions using experience and feedback.

These are not mutually exclusive: practical systems often combine expert knowledge, control, optimization, and learned policies.

The Traditional Reinforcement Learning Paradigm



Classical reinforcement learning:

- learns from consequences rather than labeled targets,
- balances exploration and exploitation,
- optimizes long-term reward.

For much of its history, reinforcement learning was framed as learning through continual interaction.

In practice, this usually means learning in a simulator before deployment.

A Data-Centric View of Reinforcement Learning

Interaction-centric RL

initiate simulator
↓
collect data
↓
update model
↓
repeat

Improve through repeated interaction.

Data-centric ML

large logged dataset
↓
scalable offline training
↓
expressive model
↓
deployment

Transformers and foundation-model-style scaling benefit from large static datasets.

What can we learn from the interaction data we already have?

Start Simple: Behavior Cloning

observed situation + operator decision → supervised learning → learned policy

- Observe what an operator or existing controller did.
- Train a model to predict the same action in similar situations.
- Deploy the trained model as a learned "policy".

Behavior cloning is fully offline and requires no simulator or reward model.

Can we do more than learn based on operator data? Can we prefer decisions that led to better long-term outcomes?

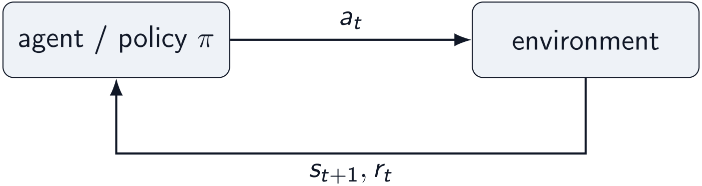
To answer that, we first need a language for decision problems: states, actions, and policies.

States, Actions, and Policies

Object	Grain-terminal example
state s_t	current queue, silo inventory, grain quality, train progress
action a_t	unload truck, choose pit, route conveyor, assign silo, fill car
policy $\pi(a \mid s)$	rule or model that chooses actions from states
next state s_{t+1}	the facility after the action has changed it

A decision problem starts with states, actions, and a policy that maps situations to decisions.

Agent-Environment Loop



- **Agent:** chooses the next action from the current state.
- **Environment:** the facility or simulation that returns the next state and reward.
- **Key contrast:** supervised learning predicts from fixed data; RL actions create the next data.

Imitation Learning: Turn Decisions Into Supervised Learning

First idea: collect observed behavior and train a policy to predict what the operator would do.

Historical demonstrations come from a behavior policy π_b, e.g. human terminal operators:

$$\mathcal{D} = \{(s_i, a_i)\}_{i=1}^{N}, \qquad a_t \sim \pi_b(a \mid s_t)$$

Train π_θ as a supervised action predictor:

Continuous actions
steering angle, conveyor speed

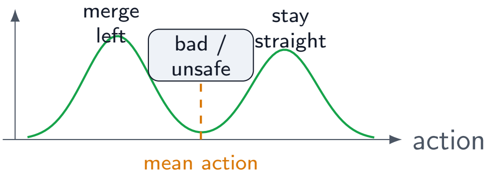
$$\min_\theta \sum_{i \in \mathcal{D}} \|a_i - \pi_\theta(s_i)\|^2$$

Discrete actions
route choice, silo assignment

$$\min_\theta \; -\sum_{i \in \mathcal{D}} \log \pi_\theta(a_i \mid s_i)$$

Attractive because it is fully offline, simple to train, and does not require defining a reward.

Pitfall 1: Averaging Demonstrations Can Fail



Consider imitation data for self-driving cars:

- in similar observed states, driver 1 stays in lane
- driver 2 changes lanes
- squared-error regression learns the conditional mean action

Problem: the mean action can lie between valid modes: a poor command that no expert demonstrated.

Pitfall 2: Errors Accumulate Over Time

t: small error

↓

$t{+}1$: shifted state

↓

$t{+}2$: bigger error

↓

$t{+}3$: unfamiliar state

A policy trained only on expert states may not know how to recover from states caused by its own mistakes.

- $\rightarrow$ Distribution shift: deployment samples from the learned policy's state distribution, not the expert's.

Problem: one-step prediction errors can look small while long rollouts become unstable.

Pitfall 3: Imitation Ignores Outcomes

Behavior cloning asks only:

“What action was taken in this situation?”

It does not ask:

“Which action led to the best long-term outcome?”

Every logged decision becomes a training example. The quality of the outcome does not change how strongly it is learned.

Problem: behavior cloning reproduces past decisions, but it cannot prefer decisions with better long-term consequences.

Rewards Measure the Consequences of Decisions

A reward turns delayed product outcomes into feedback for decisions:

$$r_t = r(s_t, a_t, s_{t+1})$$

For grain loading, rewards can combine:

- train loading progress and throughput
- contract quality satisfaction
- penalties for rehandling, congestion, and delay

Rewards tell the learner what outcomes matter, not just what actions were taken.

Long-Term Consequences

A good decision is one that leads to *high future reward*, not just a high immediate reward.

$$\text{Future Reward} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$$

where $0 < \gamma < 1$ discounts delayed rewards.
A value function predicts this quantity before making a decision:

$$V(s) \;\approx\; \text{future reward, starting from state } s$$

$$Q(s,a) \;\approx\; \text{future reward, starting from state } s \text{ and taking action } a$$

Main takeaway

V and Q quantify how good it is to be in a state and to take an action. They are central objects in RL.

Learning the Value Function

Learning the Q-function is an important step in nearly all of RL.

To keep things simple, Q satisfies a recursive relation:
long-term value = expected current reward + discounted future value.

$$Q(s_t, a_t) \approx r_t + \gamma V(s_{t+1})$$

This recursion can be solved with exact or approximate dynamic programming.

We will skip those details and use the equation only as the basic intuition behind value learning.

Three Influential Ideas in Offline Reinforcement Learning

Complementary approaches to improve behavior cloning:

1. **Reweight the logged data**
   Advantage-Weighted Regression (AWR): copy successful decisions more strongly.

2. **Learn conservative value estimates**
   Conservative Q-Learning (CQL): be pessimistic where evidence is weak.

3. **Estimate values only from supported actions**
   Implicit Q-Learning (IQL): avoid maximizing over unsupported actions altogether.

Three Common RL Families

Family	Basic idea
Value-based RL	Learn $Q(s, a)$, act greedily
Policy gradient	Directly improve $\pi(a \mid s)$
Actor-critic	Learn both policy and value

- Q-learning is conceptually simple, but is most natural when actions are discrete.
- Policy gradients apply more generally, but require expensive rollouts.
- Actor-critic methods combine both ideas: they learn a policy and a value estimate, and work with discrete or continuous actions.

Generic Actor-Critic Offline RL

Most modern actor-critic Offline RL methods separate two steps:

$$\begin{array}{l}\text{estimate } Q, V \\ \Updownarrow \\ \text{improve policy}\end{array}$$

The three ideas intervene at different points:

- **AWR** mainly changes the improve-policy step.
- **CQL** and **IQL** focus primarily on value estimation.

Policy updates need values; methods differ in how those values are made reliable.

Problem: Not All Demonstrations Are Equally Useful

Behavior cloning treats every logged decision equally.

But logged data often contains a mixture:

- strong operator decisions,
- reasonable routine decisions,
- choices that led to delay, rehandling, or poor downstream outcomes.

Question

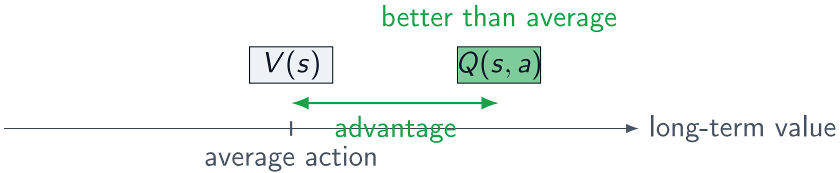
Can we learn more from the better demonstrations?

Core Idea: Advantage

Compare the logged action to what is typical in the same state:

$$A(s, a) = Q(s, a) - V(s).$$

- $Q(s, a)$ measures how good it is to take action a in state s.
- $V(s)$ measures how good the state is on average.
- Their difference says how much better this action is than expected.



This difference is called the **advantage**.

Advantage-Weighted Regression (AWR)

Behavior cloning solves:

$$\min_\theta \sum_i \ell(\pi_\theta(s_i), a_i)$$

AWR keeps the same supervised-learning loss, but weights each example:

$$\min_\theta \sum_i \exp(\beta A(s_i, a_i))\, \ell(\pi_\theta(s_i), a_i)\,.$$

Key point

The only change is the weighting. Everything else remains supervised learning.

Next question: Where do reliable estimates of Q and V come from?

Problem: Offline Values Can Be Overconfident

To compute useful weights, we need value estimates.

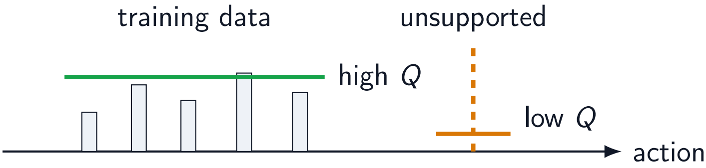
But value estimation is harder offline:

- online learning can test actions and correct bad guesses,
- offline learning cannot collect new feedback,
- unsupported actions may receive overly optimistic predicted values.

Question

How can we learn values without trusting predictions where the data is weak?

Core Idea: Conservative Value Learning



$$\mathcal{L}_{\mathrm{CQL}} = \underbrace{\mathcal{L}_{\mathrm{Bellman}}}_{\text{fit observed transitions}} + \alpha \left(\underbrace{\mathbb{E}_{a\sim\mu} Q(s,a)}_{\text{push down candidate actions}} - \underbrace{\mathbb{E}_{(s,a)\sim\mathcal{D}} Q(s,a)}_{\text{keep logged actions high}} \right).$$

Main takeaway

CQL changes only the value-learning objective: fit the data, but penalize high Q-values for unsupported actions.

A Different Idea: Never Evaluate Unseen Actions

CQL keeps the maximization, but makes unsupported actions pessimistic.

Can we avoid the maximization entirely?

Standard Q-learning uses the target:

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a').$$

The problem is that the maximization evaluates actions that never appeared in the dataset, creating extrapolation errors.

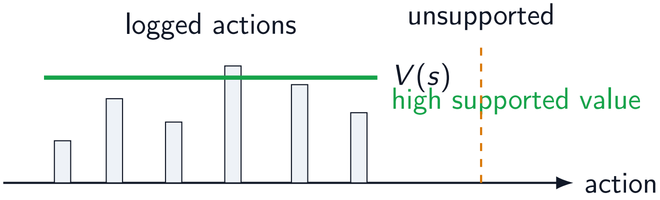
Main takeaway

IQL's key idea: never evaluate actions that are not in the dataset.

Learn the Best Supported Value

Instead of computing $\max_a Q(s, a)$, IQL learns a state value:

$$V(s) \approx \text{upper expectile of } Q(s, a) \quad \text{for logged actions } a.$$



Expectile regression is the tool used to estimate this high supported value.

Then learn Q with:

$$Q(s, a) \leftarrow r + \gamma V(s').$$

Notice: $\max_a Q(s, a)$ has disappeared.

Policy Extraction: Reuse AWR

Once $V(s)$ and $Q(s,a)$ have been learned, the policy update is familiar.

logged dataset

↓

learn V high supported value

↓

learn Q $Q(s,a) \leftarrow r + \gamma V(s')$

↓

compute $A = Q - V$

↓

run AWR $w = e^{\beta A}$

Main takeaway

IQL changes value estimation, not policy optimization: $\boxed{V \to Q \to A \to \pi}$

What We Could Cover Next

This tutorial focused on one branch: model-free Offline RL.

Other major directions:

- **Model-Based RL:** Can we learn a model and plan with it?
  World models, imagined rollouts. Examples: Dreamer, TD-MPC, MBPO.
- **Foundation Models for Robotics:** Can one policy solve many tasks?
  Large-scale imitation and vision-language-action models. Examples: RT-1, RT-2, OpenVLA, π_0.
- **Sequence Modeling:** Can we replace Bellman updates with sequence modeling?
  Trajectories as sequences, Transformer policies. Examples: Decision Transformer, Trajectory Transformer.
- **Offline-to-Online Adaptation:** How do we keep improving after deployment?
  Start from logged data, then fine-tune with limited online interaction.

Outlook: Offline RL is increasingly one component of larger data-driven decision-making systems.

Summary: What Changes Across Methods?

Method	Main contribution	Role
AWR	Advantage-weighted policy improvement	Policy update
CQL	Conservative value estimation	Value learning
IQL	In-sample value estimation	Value learning

Main message

Modern Offline RL largely consists of estimating reliable values from logged data, then improving the policy using those values.

Different methods primarily differ in how reliable values are obtained, not in the overall objective.

References

Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song.
Diffusion policy: Visuomotor policy learning via action diffusion.
arXiv preprint arXiv:2303.04137, 2023.

Philippe Hansen-Estruch, Ilya Kostrikov, Michael Janner, Jingyun G. Chen, and Sergey Levine.
IDQL: Implicit q-learning as an actor-critic method with diffusion policies.
arXiv preprint arXiv:2304.10573, 2023.

Ilya Kostrikov, Ashvin Nair, and Sergey Levine.
Offline reinforcement learning with implicit q-learning.
arXiv preprint arXiv:2110.06169, 2021.

Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine.
Conservative q-learning for offline reinforcement learning.
In *Advances in Neural Information Processing Systems*, 2020.

Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu.
Offline reinforcement learning: Tutorial, review, and perspectives on open problems.
arXiv preprint arXiv:2005.01643, 2020.

Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine.
Advantage-weighted regression: Simple and scalable off-policy reinforcement learning.
arXiv preprint arXiv:1910.00177, 2019.

Richard S. Sutton and Andrew G. Barto.
Reinforcement Learning: An Introduction.
MIT Press, 2 edition, 2018.